

below:

```
(s-pad-left 3 "0" "42")
"042"
```

```
(s-pad-right 3 "." "45")
"45."
```

Shorter string functions

Let us go through a few functions that return a shorter string from the input string. The *truncate* function takes a length argument, and returns the input string for the specified length. For example:

```
(s-truncate 6 "What do you think?")
"Wha..."
```

The definition of the *s-truncate* function from the *s.el* library is as follows:

```
(defun s-truncate (len s &optional ellipsis)
  "If S is longer than LEN, cut it down and add ELLIPSIS to
the end.
The resulting string, including ellipsis, will be LEN
characters
long.
When not specified, ELLIPSIS defaults to '...'."
  (declare (pure t) (side-effect-free t))
  (unless ellipsis
    (setq ellipsis "..."))
  (if (> (length s) len)
    (format "%s%s" (substring s 0 (- len (length
ellipsis))) ellipsis)
    s))
```

You can request for the first length of the characters from the left or the right end of the string using the *s-left* and *s-right* functions, respectively. A couple of examples are given below:

```
(s-left 3 "Never mind")
"Nev"
(s-right 4 "/etc/resolv.conf")
"conf"
```

The *s-chop-suffix* function takes a suffix argument, and removes it from the input string before returning the result. Similarly, the *s-chop-prefix* function accepts a prefix argument and returns the string without the prefix as shown below:

```
(s-chop-suffix ".js" "main.js")
"main"

(s-chop-prefix "/" "/root")
"root"
```

Longer string functions

In this section, we shall go through functions that can create larger strings. The *s-repeat* functions take two arguments, a count and a character, and return a string that has count occurrences of the character. For example:

```
(s-repeat 6 "*")
"*****"
```

You can concatenate multiple strings using the *s-concat* function as shown below:

```
(s-concat "pqr" "stu" "vwx" "yz")
"pqrstuvwxyz"
```

The *s-prepend* function prepends a prefix to a string, while the "s-append" function appends the suffix to the string as illustrated below:

```
(s-prepend "abc" "def")
"abcdef"

(s-append "bar" "foo")
"foobar"
```

List operations

We shall now go through a few list based operations. A string that contains newlines can be split individually using the *s-lines* function as follows:

```
(s-lines "1 2 3\n4 5 6\n7 8 9")
("1 2 3" "4 5 6" "7 8 9")
```

The *s-match* function takes a regular expression and a string, and checks to see if the expression is present in the string. The string that is matched is returned if present, otherwise 'nil' is returned.

```
(s-match "^123" "12345")
("123")

(s-match "^45" "12345")
nil
```

You can slice an input string at a character using the *s-slice-at* function. Few examples are shown below:

```
(s-slice-at "\\." "100.45")
("100" ".45")

(s-slice-at "\\." "10045")
("10045")
```